Assessment–II : Real-Time Tower Health Monitoring in Telecom Industry Using Kafka

Business Problem

Our assessment focuses on the Telecommunication Industry and how they utilise data streaming in their everyday operations. Telecom companies like Jio, Airtel and Vodafone heavily depend on the network towers that they operate across India. Since these towers are responsible for transmitting vast amount of data every second, a sudden drop in their signal strength can lead to dropped calls, slow internet or service outage which can cause user inconvenience and may result in customer churn and loss of brand trust.

For our case, we have assumed a newly launched and rapidly growing Telecom company "Bharat Connect" that operates in limited cities in India. Being in their initial phase, Bharat Connect is facing trouble with tower signal drops and are receiving frequent complaints regarding the same. Thus, real-time monitoring of tower health becomes crucial for a new entrant in order to survive in the Telecom industry alongside big players like Jio and Airtel, that is why, for better user experience, Bharat Connect has decided actively respond to the towers experiencing signal drops, Bharat Connect has decided to implement a real-time surveillance system that continuously monitors tower signal health and triggers an alert to the company office whenever the signal drops below a certain threshold.

We have created a Kafka pipeline for data streaming of tower signal alert, following are the pipeline's components:

Producer

In real world scenario, the source for actual streaming data would be the network towers, transmitting vast amount of data every second. However, in our simulation, the data source is a cdr_data.csv file containing 1009 rows of information for all cities combined where Bharat Connect operates.

The Producer is responsible for pushing each row's data into the Kafka cluster, think of it as an on-field agent constantly reporting the status of the towers.

Broker

Here, the pushed data is stored temporarily, and it lets information go between those who send it (Producer) and those who receive it (Consumers). The broker receives information from hundreds of towers without choking. Even if one broker fails, the Kafka pipeline does not crash, rather the operations are transferred to another working broker by Zookeeper.

At Bharat Connect, the broker ensures that all tower health reports whether normal or critical, are never lost, never duplicated, and always delivered.

Topic

After the broker gets the data, it stores the data into a specific Topic to keep things organized, for example, in this case the topic is created for tower alert (cdr_raw). For example, Bharat Connect can also create various other Topics as well for related information like call logs, recharge patterns etc.

Partitions

Each Topic is further divided into smaller parts called Partitions for parallel data processing in order to increase scalability. Different consumers can read information from different partitions at the same time without any interference.

Though this component is a part of the Kafka pipeline, it has not been used in our simulation, we are only operating on a single partition for data consumption.

Stream and Storage

The tower health readings are continuously streamed and are stored in the MongoDB database as per the trigger. This allows Bharat Connect to do both real time inspection as well as analysis if historically stored data, crucial for their long-term efficiency.

Consumer/Consumer Groups

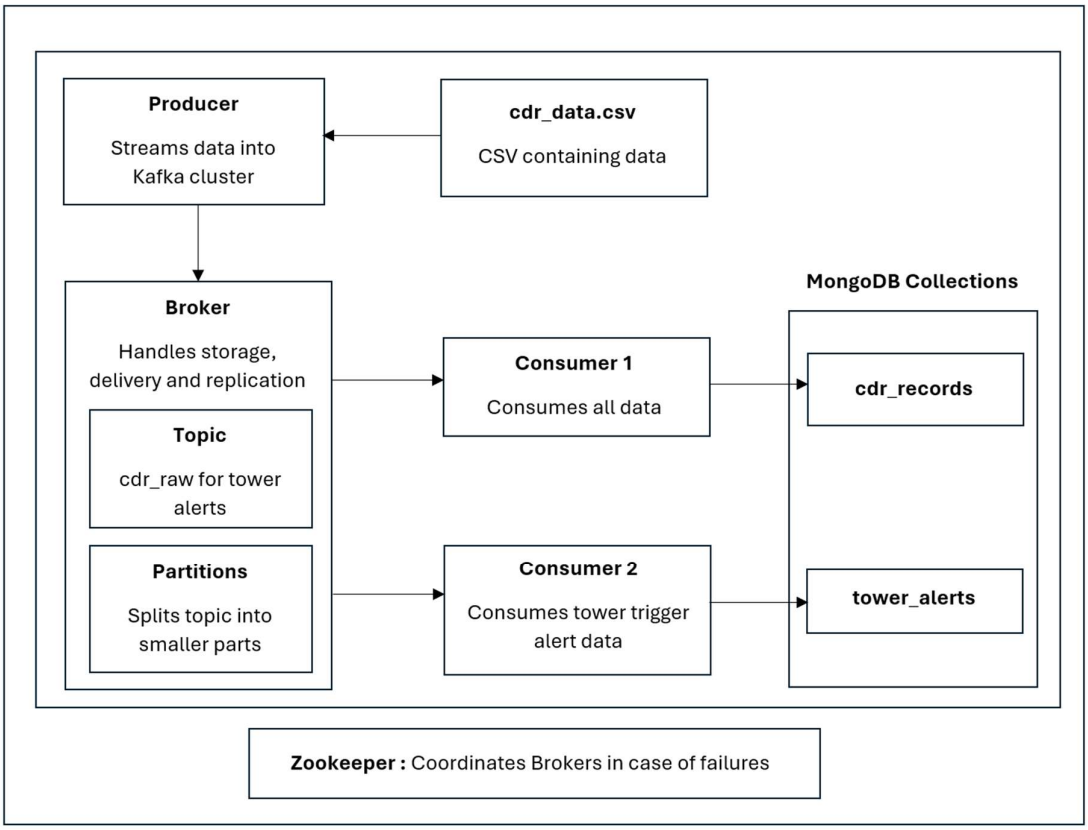
On the receiving end of this data stream is the Consumer that work like the network engineer continuously scanning the logs for tower alerts.

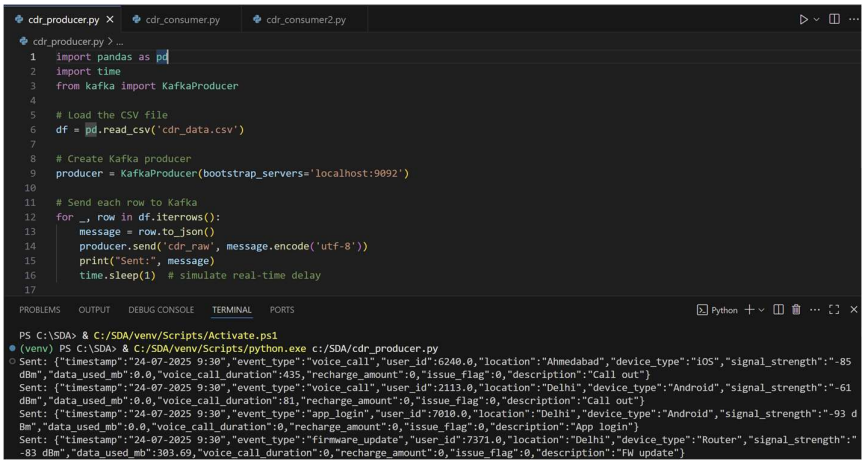
Our simulation has two Consumers, one that is responsible for consuming and pushing entire data from Kafka into MongoDB. Second consumer is trigger based, responsible for consuming only the relevant information (logs with tower_alert as event type) and pushing them to a separate MongoDB collection.

Zookeeper

Behind the scenes, Zookeeper works quietly, managing the entire Kafka pipeline. If a broker fails, it detects the failure immediately and switches the responsibility to another broker, thus ensuring active streaming.

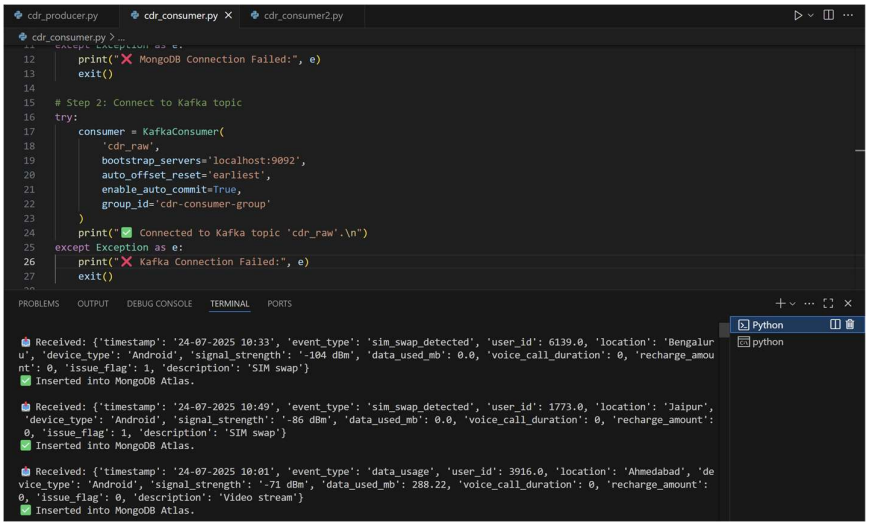
Data Pipe Line- Flow Chart & Working





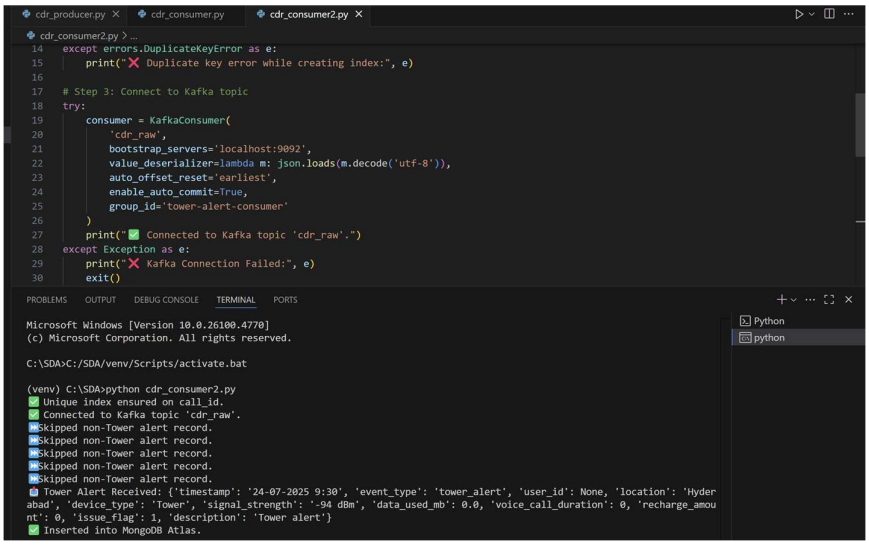
Producer

Streaming Telecom data into Kafka cluster



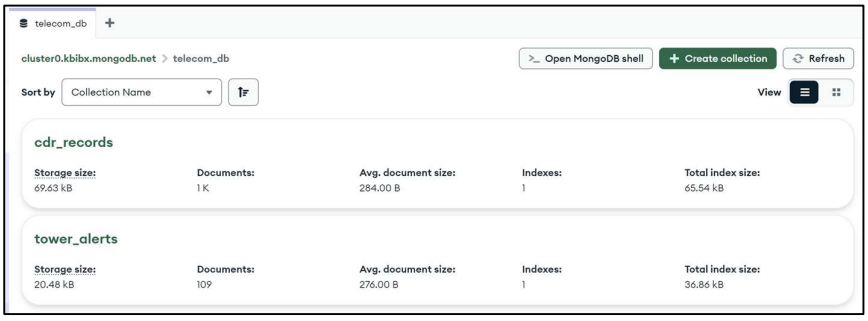
Consumer-1

Receiving and sending all data into MongoDB



Consumer-2

Receiving and sending triggered data into MongoDB



Database

MongoDB with two separate collections.